

Report 06

LangGraph Orchestrator & LLM-Free Fallback

Project:	JalJeev — Agentic Marine Intelligence System
Phase:	Phase 0, Week 3–4 (Sep 8–20, 2026)
Focus:	Agentic planner, state machine design, and LLM-free fallback

1. Objective

Build the orchestration layer that takes a natural-language question (*"Is it safe to fish near Puri tomorrow?"*), decomposes it, routes it to the right agents, collects their responses, and synthesises a coherent, evidence-backed answer — while ensuring the entire system works **without any LLM at all**.

2. LangGraph State Machine

2.1 The Graph

`parse_intent` → `resolve_location` → `execute_tools` → `synthesize_answer`

Four nodes, strictly sequential, no branches or cycles:

Node	Job	LLM Required?
<code>parse_intent</code>	NL → structured {intent, lat, lon, vessel_class, time_hmz}	Yes (Groq) or No (rule-based)
<code>resolve_location</code>	Text name → coordinates (geocoding via Nominatim)	No
<code>execute_tools</code>	Fan out to Weather, Ocean, Geo, Risk, Route agents	No — pure Python
<code>synthesize_answer</code>	Evidence receipt → natural-language answer	Yes (Groq) or No (template)

2.2 Key Design: The LLM Does Exactly Two Jobs

- 1. **Parse intent** — translate natural language to structured parameters
- 2. **Synthesize answer** — phrase a final answer from a pre-computed evidence receipt

Everything between those two nodes is deterministic Python and SQL. No agent asks a model to compute or estimate a physical value. This is not a preference — it is a safety requirement.

2.3 Tool Dispatch

After intent parsing, the planner decides which agents to invoke based on the parsed intent type:

```
TOOL_MAP = {
    "safety_check": ["weather", "ocean", "geo", "risk"],
    "pfz_query":    ["weather", "ocean", "geo", "fishing", "risk"],
    "route_plan":   ["weather", "ocean", "geo", "risk", "route"],
```

```
"passage_plan": ["weather", "ocean", "geo", "risk", "passage"],  
"what_if":      ["weather", "ocean", "geo", "risk", "simulation"],  
"anomaly_query": ["weather", "ocean", "anomaly"],  
"general":      ["weather", "ocean", "geo"],  
}
```

All selected agents run **concurrently** via `asyncio.gather` with per-agent timeouts.

3. LLM Integration: Gemini → Groq Migration

3.1 The Gemini Incident (Aug 26, 2026)

The system originally used Google Gemini as the primary LLM. What happened:

1. Development testing exhausted Gemini's **20-requests/day** free tier quota
2. Every subsequent call returned an **instant 429 RESOURCE_EXHAUSTED**
3. But this exception was being silently swallowed by a bare except Exception: return None
4. Symptom: ~20-second stalls that looked like a hang or async bug
5. **Two full debugging sessions** were spent investigating async timeouts, network issues, and thread pool exhaustion
6. **Root cause:** One line printing `type(exc).__name__` found it immediately

3.2 The Standing Rule

An adapter or LLM call must never swallow an exception without recording its type and message.
"It's probably just slow" is a guess, not a diagnosis.

3.3 Groq as Replacement

Groq (openai/gpt-oss-120b) replaced Gemini as primary LLM:

- More generous free tier
- Measured ~0.6s for a trivial call
- Same structured output protocol (Pydantic-forced JSON)

4. The Rule-Based Fallback

4.1 Why It Exists

Groq is still a free-tier third-party API. Eventually its quota will be exhausted too. The rule-based fallback exists because:

- The safety-critical path must be reachable with zero LLM calls
- An unreachable API at demo time must not be a showstopper
- A system that works without an LLM proves the LLM adds convenience, not capability

4.2 Rule-Based Intent Parser

```
INTENT_PATTERNS = [
```

```

(r"safe|danger|risk|can i go", "safety_check"),
(r"fish|pfz|catch|trawl", "pfz_query"),
(r"route|safest path|navigate", "route_plan"),
(r"passage|port.to.port|kochi.to", "passage_plan"),
(r"what if|simulate|change", "what_if"),
(r"anomal|decline|unusual|trend", "anomaly_query"),
]

```

Regex + keyword matching. Disambiguates using priority ordering and context clues. Falls back to "general" if nothing matches.

4.3 Rule-Based Answer Renderer

```

def render_answer(receipt: EvidenceReceipt) -> str:
    lines = []
    lines.append(f"Risk Level: {receipt.risk_level}")
    lines.append(f"Risk Score: {receipt.risk_score}/100")
    if receipt.vetoed:
        lines.append(f"■ REJECTED: {' '.join(receipt.veto_reasons)}")
    for factor, points in receipt.factor_breakdown.items():
        lines.append(f"  {factor}: +{points}")

```

This renderer is structurally incapable of hallucinating — it only repeats fields already present in the receipt.

4.4 Verified End-to-End

The rule-based path was verified live with **zero LLM calls**: geocoding, all 5 marine agents, risk scoring, evidence aggregation, and answer rendering — all functioning correctly.

5. Timeout Architecture

5.1 Per-Agent Timeouts (Not Per-Gather)

One slow source must not discard fast successful results. Each agent gets a 25-second individual timeout:

```
async def _run_with_timeout(agent_fn, name, timeout=25):
    try:
        return await asyncio.wait_for(agent_fn(), timeout=timeout)
    except asyncio.TimeoutError:
        return partial_result(f"{name} timed out after {timeout}s")
```

5.2 Total Query Timeout

The entire /chat endpoint has a 115-second ceiling to prevent runaway requests.

6. Session Memory

The Memory Agent provides multi-turn conversational context:

- Stores recent queries and responses in Redis with a 1-hour TTL
- Enables follow-up queries: "What about tomorrow?" (inherits the previous location)
- Context window: last 5 exchanges

7. Impact: Innovation #12

"The LLM is optional, not a dependency. Remove the API key entirely and the system still answers, with a renderer that is structurally incapable of hallucinating because it only repeats fields from a receipt."